

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_ARGS 64
#define BUF_SIZE 512

// 1. ESCAPE SEQUENCE PARSER: Treats '\\' as an escape character
int parse_escaped_input(const char *input, char *args[]) {
    int arg_count = 0;
    int len = strlen(input);
    int i = 0;

    while (i < len) {
        // Skip leading whitespace
        while (i < len && isspace(input[i])) i++;
        if (i >= len) break;

        char buf[BUF_SIZE];
        int buf_idx = 0;

        while (i < len && !isspace(input[i])) {
            // Handle Backslash Escape Sequences (\ , \|, \>, etc.)
            if (input[i] == '\\' && i + 1 < len) {
                i++; // Skip backslash
                buf[buf_idx++] = input[i++]; // Store escaped character literally
            } else {
                buf[buf_idx++] = input[i++];
            }
        }

        buf[buf_idx] = '\0';
        args[arg_count++] = strdup(buf);
    }
}
```

```

    args[arg_count] = NULL; // Null-terminate argument array for execvp
    return arg_count;
}

// 2. PROCESS LAUNCHER: Spawns child processes using fork() and execvp()
void execute_command(char *args[]) {
    if (args[0] == NULL) return; // Empty command

    pid_t pid = fork();

    if (pid < 0) {
        // Forking Error
        perror("Fork failed");
        return;
    } else if (pid == 0) {
        // Child Process: Execute program
        if (execvp(args[0], args) == -1) {
            // Execution Error Handling
            perror("Execution error");
        }
        exit(EXIT_FAILURE);
    } else {
        // Parent Process: Wait for child to complete
        int status;
        waitpid(pid, &status, 0);
    }
}

void free_args(char *args[], int count) {
    for (int i = 0; i < count; i++) {
        free(args[i]);
        args[i] = NULL;
    }
}

int main() {

```

```

char input_buf[BUF_SIZE];

while (1) {
    printf("\nExecutor Shell > ");
    if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

    // Remove newline
    input_buf[strcspn(input_buf, "\n")] = 0;

    if (strcmp(input_buf, "exit") == 0) break;

    char *args[MAX_ARGS];
    int arg_count = parse_escaped_input(input_buf, args);

    if (arg_count > 0) {
        // Debug Parser Output
        printf("--- Debug: Parsed Command Arguments ---\n");
        for (int i = 0; i < arg_count; i++) {
            printf("Arg [%d]: \"%s\"\n", i, args[i]);
        }

        // Launch Child Process
        printf("--- Execution Output ---\n");
        execute_command(args);

        free_args(args, arg_count);
    }
}

return 0;
}

```

OUTPUT :

```
Executor Shell > echo Hello\ \ World
--- Debug: Parsed Command Arguments ---
Arg [0]: "echo"
Arg [1]: "Hello |"
Arg [2]: "World"
--- Execution Output ---
Hello | World

Executor Shell > invalidcmd123
--- Debug: Parsed Command Arguments ---
Arg [0]: "invalidcmd123"
--- Execution Output ---
Execution error: No such file or directory

Executor Shell > exit
abhis@MIKHAIL-37:~$ valgrind --leak-check=full ./executor
==772== Memcheck, a memory error detector
==772== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==772== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==772== Command: ./executor
==772==

Executor Shell > exit
==772==
==772== HEAP SUMMARY:
==772==     in use at exit: 0 bytes in 0 blocks
==772==   total heap usage: 2 allocs, 2 frees, 2,048 bytes allocated
==772==
==772== All heap blocks were freed -- no leaks are possible
==772==
==772== For lists of detected and suppressed errors, rerun with: -s
==772== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```